DEVOPS REAL-TIME INTERVIEW Q&A
🟢 **PART-1 — DAILY WORK + INCIDENT MANAGEMENT**
Q1 → Q20 | Zero Knowledge → Real-Time Job Level

1️⃣ DAILY WORK & TICKET HANDLING

---

1. How do you pick tickets?

🧠 **Simple Meaning**
Company lo developers, QA, customers, monitoring tools nunchi different problems vastayi.
Vatini **tickets** antam.
Example:
Ticket 1 → Website down
Ticket 2 → Login issue for 5 users
Ticket 3 → Change server name
Ticket 4 → Production API returning 500
Anni okesari solve cheyyalem.
So first **most important issue** ni pick chestham.

🎯 **Interview Answer**
"I prioritize tickets based on business impact, severity, number of affected users, and urgency. I usually handle production-critical issues first, followed by high-priority tasks and then normal requests."

🔥 **Real-Time Example**
Suppose morning 10 AM ki 5 tickets vachayi:
P1 → Production website down
P2 → API slow
P3 → Developer needs access
P3 → Disk cleanup
P4 → Documentation update
Nenu first:
P1 Website Down
↓
Investigate
↓
Restore service
↓
Monitor
↓
P2 API issue
↓
P3
↓
P4

🛠️ **Actual DevOps Work**
Jira/ServiceNow open chestha.
Ticket details check:
Priority
Impact
Affected service
Environment
Error
Requester
Deadline
Then priority decide chestha.

⚠️ **Interview Point**
"I always work on tickets in the order they came" ani cheppakudadhu.
Correct:
Impact + urgency + priority based on work chestha.

2️⃣ What is P1/P2?

🧠 **Simple Meaning**
P = **Priority**
Usually:
P1 → Critical
P2 → High
P3 → Medium
P4 → Low

🔥 **P1 Example**
Production website completely down.
Users → ❌ Website
Business completely affected.
So:

P1

🔥 **P2 Example**

Website working undi, but payment API slow ga undi.

Website → ✅

Payment → ⚠️ Slow

Important issue but complete outage kaadu.

So:

P2

🎯 **Interview Answer**

"P1 is a critical production issue with major business or user impact, such as a complete service outage. P2 is a high-priority issue that has significant impact but does not completely stop the service."

⚠️ **Important**

Priority definitions **company-to-company change avvachu**.

Interview lo:

"The exact priority definitions depend on the organization's incident policy."

ani chepthe professional ga untundi.

3️⃣ **How many tickets do you handle daily?**

🧠 **Simple Meaning**

Interviewer actually number kanna **how you manage workload** ani test chestunnadu.

❌ **Bad Answer**

"I handle exactly 10 tickets every day."

Why?

Real DevOps work lo one production incident itself 5 hours take cheyyachu.

🎯 **Better Interview Answer**

"It depends on the complexity and priority of the tickets. On a normal day I may handle several small requests or a few major incidents. During a critical production incident, I focus primarily on restoring the service rather than ticket count."

🔥 **Real-Time**

Normal day:

3 small tickets

2 deployment tasks

1 monitoring issue

2 access requests

But production outage vachindi:

Production outage

↓

Stop normal work

↓

Incident investigation

↓

Restore service

↓

RCA

⚠️ **Interview Point**

Ticket count kanna impact and resolution quality important.

4️⃣ **What is a ticket lifecycle?**

🧠 **Simple Flow**

Ticket creation nunchi closure varaku:

OPEN

↓

ASSIGNED

↓

IN PROGRESS

↓

BLOCKED (if needed)

↓

RESOLVED

↓

CLOSED

🔥 **Real-Time Example**

Developer:

"Production deployment failed."

Ticket create chestadu.

OPEN

DevOps engineer ticket accept chestadu:

IN PROGRESS

Pipeline logs check chestadu.

Issue fix chestadu.

Testing chestadu.
RESOLVED
Requester verify chesaka:
CLOSED

🎯 **Interview Answer**

"A ticket generally starts in Open state, gets assigned and moved to In Progress, may become Blocked if there is a dependency, then moves to Resolved after the fix is verified, and finally Closed after confirmation."

5️⃣ **How do you track your work?**

🧠 **Simple Meaning**

Manam chesina work marchipokunda tracking system use chestham.
Common tools:
Jira
ServiceNow
Azure Boards
GitHub Issues

🔥 **Real-Time**

Suppose ticket:
DEVOPS-2456
Production API issue
Daily update:
10:00 AM → Started investigation
11:00 AM → Found high CPU
12:00 PM → Scaled service
01:00 PM → Monitoring stable
02:00 PM → Resolved

🎯 **Interview Answer**

"I track my work through Jira or ServiceNow. I keep ticket status, investigation notes, blockers, actions taken, and resolution details updated so the team has clear visibility."

⚠️ **Professional Point**

Ticket lo **"fixed"** ani matrame rayakunda:
What happened?
What did I check?
What was the root cause?
What action was taken?
What is the current status?
document cheyyali.

6️⃣ **What do you do if a ticket is blocked?**

🧠 **Simple Meaning**

Nuvvu work cheyyali anukuntunnav.
But another team dependency undi.
Example:
DevOps
↓
Needs DB access
↓
DBA approval required
↓
BLOCKED

🔥 **Real-Time**

Ticket:
"Deploy application to production."
But production credentials/access unavailable.
Nuvvu:
Check access
↓
Confirm missing permission
↓
Contact IAM/Cloud team
↓
Update Jira
↓
Wait for access
↓
Continue deployment

🎯 **Interview Answer**

"If a ticket is blocked, I first identify the exact dependency, communicate with the responsible team, document the blocker in the ticket, and escalate when necessary so that the work can continue."

⚠️ **Important**

Blocked ticket ni silent ga vadileyakudadhu.
Communicate + document + escalate.

7️⃣ How do you prioritize tasks?

🧠 Simple Formula

Priority decide cheyyadaniki:
Impact
+
Urgency
+
Business importance
+
Number of users affected
+
SLA
+
Dependencies

🔥 Example

Task A → Production down
Task B → Developer wants new server
Task C → Documentation
Order:
A
↓
B
↓
C

🎯 Interview Answer

"I prioritize tasks based on severity, business impact, urgency, affected users, SLA requirements, and dependencies. Production-critical issues always take precedence over routine tasks."

2️⃣ INCIDENT MANAGEMENT & RCA

8️⃣ What is Incident Management?

🧠 Simple Meaning

Production lo problem vachindi.
Goal:
First service ni restore cheyyali.
Root cause later detailed ga investigate cheyyachu.

🔥 Example

Website:
Users
↓
Load Balancer
↓
EC2
↓
Application
Suddenly:
HTTP 500
Incident management flow:
Alert
↓
Acknowledge
↓
Investigate
↓
Mitigate
↓
Restore
↓
Monitor
↓
RCA

🎯 Interview Answer

"Incident management is the process of detecting, responding to, mitigating, and resolving production issues while minimizing business and user impact."

⚠️ Key Difference

Incident management = restore service quickly
RCA = understand why it happened

9️⃣ **What is RCA?**
RCA = **Root Cause Analysis**
🧠 **Simple Meaning**
Problem vachindi.
Question:
"Asalu ee problem enduku vachindi?"
Example:
Website down.
Immediate observation:
EC2 CPU = 100%
But CPU 100% itself root cause kaakapovachu.
Further investigation:
CPU 100%
↓
Application process
↓
Memory leak
↓
Traffic increased
↓
Application bug
Actual root cause:
Application memory leak under high traffic.
🎯 **Interview Answer**
"RCA is the process of identifying the underlying cause of an incident rather than only fixing the visible symptom. The goal is to prevent the issue from happening again."
🔥 **RCA lo usually:**
What happened?
When happened?
Who/what affected?
Why happened?
How was it detected?
What fixed it?
How do we prevent recurrence?

🔟 **How do you handle pressure in production?**
🧠 **Simple Meaning**
Production outage ante panic avvakudadhu.
❌ **Bad approach**
Server down!
Restart everything!
Delete pod!
Change random config!
This can make problem worse.
✅ **Correct approach**
Stay calm
↓
Understand impact
↓
Check monitoring
↓
Follow runbook
↓
Mitigate
↓
Communicate
↓
Monitor
↓
RCA
🎯 **Interview Answer**
"During production incidents, I stay calm and follow the incident response process. I first understand the impact, check monitoring and logs, apply the safest mitigation, communicate regularly with stakeholders, and then perform RCA after service restoration."
⭐ **Strong Interview Line**
"During a critical incident, my first priority is service restoration, followed by root-cause analysis and preventive actions."

1️⃣1️⃣ **Give an example of a critical incident you handled.**
🧠 **Interviewer expects STAR style**
S → Situation

T → Task
A → Action
R → Result

🔥 **Example**

Situation:
Production API suddenly started returning HTTP 500 errors.

Task:
My responsibility was to restore the service quickly.

Action:
CloudWatch
↓
Checked CPU/memory
↓
Checked ALB target health
↓
Checked application logs
↓
Found unhealthy instances
↓
Removed unhealthy targets
↓
Scaled healthy instances

Result:
Traffic recovered and error rate returned to normal.

🎯 **Interview Answer**

"In one production incident, the application started returning 500 errors. I checked CloudWatch metrics, ALB target health, and application logs. I identified unhealthy instances, removed them from serving traffic, scaled healthy capacity, and monitored the service until the error rate returned to normal. Afterward, I documented the RCA and preventive actions."

⚠️ **Only claim incidents you can actually explain technically.**

1️⃣2️⃣ What tools do you use to monitor incidents?

🧠 **Main categories**

Type	Tools
Metrics	Prometheus, CloudWatch
Dashboards	Grafana
Logs	ELK, CloudWatch Logs
Kubernetes	kubectl
Alerts	Alertmanager, CloudWatch
APM/Tracing	OpenTelemetry, etc.

🔥 **Kubernetes Example**

kubectl get pods
Then:
kubectl logs <pod>
Then:
kubectl describe pod <pod>

🎯 **Interview Answer**

"I use CloudWatch for AWS monitoring and Prometheus with Grafana for Kubernetes and application metrics. For incidents, I correlate metrics, logs, events, and traces to identify the problem."

1️⃣3️⃣ How do you tell stakeholders about incidents?

🧠 **Simple Meaning**

Technical problem ni business people ki technical jargon lo cheppakudadhu.

❌ **Bad**

"Pod eviction occurred because of node pressure."
Stakeholder ki maybe ardham kaadu.

✅ **Better**

"The application is currently experiencing elevated errors. We identified the affected infrastructure and are working on restoring normal service."

🔥 **Incident update structure**

What happened?
↓
Impact?
↓
What are we doing?
↓
Current status?
↓

Next update?

🎯 **Interview Answer**

"I communicate the impact, current status, actions being taken, and expected next update in clear language. I avoid unnecessary technical details and keep stakeholders updated throughout the incident."

1️⃣4️⃣ How do you prevent incidents in production?

🧠 **Simple Meaning**

Problem vachaka fix cheyyadam okati.

Problem **malli raakunda prevent cheyyadam** important.

🔥 **Prevention methods**

Monitoring
+
Automated testing
+
Code review
+
Infrastructure as Code
+
Health checks
+
Deployment strategy
+
Backups
+
Capacity planning
+
Post-mortems

Example

Repeated disk-full issue.

First time:

Clean disk

But prevention:

Disk monitoring
↓
80% warning
↓
90% critical
↓
Alert
↓
Log rotation

🎯 **Interview Answer**

"I prevent incidents through proper monitoring, health checks, testing, controlled deployments, capacity planning, automation, and learning from previous incidents through RCA and post-mortems."

1️⃣5️⃣ What is an On-Call Rotation?

🧠 **Simple Meaning**

Production issues 24/7 రావచ్చు.

Everyone 24/7 awake undaleru 😄.

So team members turns lo production support handle chestaru.

Example:

Monday → Ravi
Tuesday → Suresh
Wednesday → Arun
Thursday → Priya

🔥 **Incident**

2 AM:

Production alert
↓
On-call engineer
↓
Acknowledges alert
↓
Investigates

🎯 **Interview Answer**

"On-call rotation is a scheduled support model where engineers take turns being responsible for production incidents outside normal working hours. It ensures continuous operational coverage."

⚠️ **Important**

On-call engineer ante **all problems ni alone solve cheyyali** ani kaadu.

Needed teams ni involve cheyyachu.

1️⃣6️⃣ Alert vs Incident?

🧠 **Very Important Interview Question**

Alert

Monitoring system cheppindi:
"Something may be wrong."
Example:
CPU > 80%
Alert.
But application perfectly working undochu.

Incident

Actual user/business impact:
Website unavailable
Incident.

Flow

Metric abnormal
↓
ALERT
↓
Engineer investigates
↓
Actual service impact?
↓
YES
↓
INCIDENT

🎯 **Interview Answer**

"An alert is a notification generated by a monitoring system when a defined condition is detected. An incident is a confirmed event that requires response because it impacts or threatens service availability, performance, or users."

🔥 **INTERVIEW QUICK REVISION — Q1–Q16**

Ticket
↓
Prioritize
↓
P1/P2
↓
Work on ticket
↓
Incident occurs
↓
Detect
↓
Acknowledge
↓
Investigate
↓
Mitigate
↓
Restore
↓
Monitor
↓
RCA
↓
Prevent recurrence

3️⃣ GIT & JENKINS

1️⃣7️⃣ What is Git used for?

🧠 **Simple Meaning**

Git = **Code changes ni track cheyyadaniki version control system.**

Developer code change chestadu:
v1
↓
v2
↓
v3
Git history maintain chestundi.

🔥 **Example**

Developer accidentally:

Production code break
Git lo previous stable commit undi.
git log
Previous commit identify cheyyachu.

🎯 **Interview Answer**

"Git is a distributed version control system used to track source-code changes, collaborate across developers, manage branches, and revert to previous versions when required."

Common commands

git clone
git status
git add
git commit
git push
git pull
git branch
git merge
git log

1️⃣8️⃣ **What is a branch in Git?**

🧠 **Simple Meaning**

Main production code:
main
New feature kosam separate branch:

```
main
 |
 └── feature/login
```

Developer feature branch lo work chestadu.
Main code safe ga untundi.

🔥 **Example**

git checkout -b feature/payment
Work complete:

```
feature/payment
      ↓
Pull Request
      ↓
Code Review
      ↓
Merge
      ↓
main
```

🎯 **Interview Answer**

"A Git branch is an independent line of development used to work on features, fixes, or releases without directly affecting the main branch."

1️⃣9️⃣ **How do you fix merge conflicts?**

🧠 **Simple Meaning**

Two developers same file same lines change chesaru.
Git confusion:
Developer A → version A
Developer B → version B
Git asks:
"Which one should I keep?"

🔥 **Example**

git pull origin main
Conflict:

```
<<<<<<< HEAD
Developer A code
=======
Developer B code
>>>>>>> main
```

We manually decide correct code.
Then:
git add .
git commit
git push

🎯 **Interview Answer**

"I first identify the conflicting files, review both changes, resolve the conflict based on the intended functionality, run tests, and then commit and push the resolved changes."

⚠️ **Important**

Conflict resolve chesi **testing lekunda push cheyyakudadhu.**

2️⃣0️⃣ What is Jenkins used for?

🧠 Simple Meaning

Jenkins = **Automation server**

Manual process:

Developer writes code
↓
Build manually
↓
Test manually
↓
Create artifact
↓
Deploy manually

Jenkins automate chestundi:

Git
↓
Jenkins
↓
Build
↓
Test
↓
Artifact
↓
Deploy
↓
Monitor

🔥 Real-Time Example

Developer:
git push

Webhook Jenkins ki signal pampistundi.

Jenkins:

Checkout code
↓
Build
↓
Unit tests
↓
Docker image build
↓
Push image
↓
Deploy Kubernetes
↓
Health check

🎯 Interview Answer

"Jenkins is a CI/CD automation server. We use it to automate source-code checkout, build, testing, artifact creation, and deployment processes."

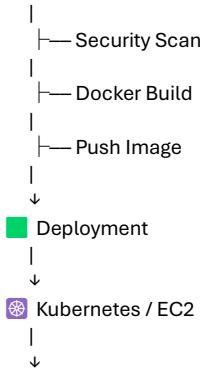
⭐ Strong Interview Answer

"In a typical pipeline, a developer pushes code to Git, Jenkins is triggered, the pipeline builds and tests the application, creates an immutable artifact such as a Docker image, and deploys it to the target environment after required validations or approvals."

🔥 Q17–Q20 REAL-TIME FLOW

Remember this one diagram:

👨‍💻 Developer
|
| git push
↓
🟦 Git Repository
|
| webhook
↓
🟧 Jenkins
|
├── Checkout
|
├── Build
|
├── Test



If interviewer asks:
"Explain your DevOps workflow."
You can say:
"Developers commit and push code to Git. Jenkins is triggered through a webhook. The pipeline checks out the code, builds the application, runs tests and security checks, creates the deployment artifact, and deploys it to the target environment. After deployment, we verify health using monitoring, logs, and application checks. If a critical issue occurs, we rollback to the last stable version."

⭐ **Next sequence**
Nee list exact order lo continue chesthe next batches:
Q21–35 → Jenkins + Docker + Kubernetes
Q36–55 → AWS + Terraform
Q56–75 → Monitoring + Release/Deployment
Q76–101 → Security + Performance + DR
Q102–130 → Advanced Kubernetes + AWS + Jenkins
Q131–160 → Logs + Tracing + Security + DevOps Practices
Q161–200 → Networking + Observability + Cost + Agile
Q201–250 → Advanced 4+ Years Real-Time
Q251–300 → Production Incident Handling
Q301–350 → Scenario-Based Interview Answers
Important: Nee original answers lo unna konni statements ni kuda real-world interview standard ki correct chestha. Example: **DNS switch immediate kaadu**, **Security Group explicit DENY support cheyyadu**, **Kubernetes Secret automatically encrypted/fully secure ani assume cheyyakudadhu**, **Terraform state rollback = simply old state file restore kaadu**. Ilaanti subtle points ni correct version lo explain chestha, so interview lo wrong answer cheppakunda untav.